54

身份认证

本章作者：**Peter Reiher（UCLA）**

54.1 引言

我们要应对各式各样的安全目标，以及为实现这些目标而制定的安全策略。那么，我们需要操作系统做什么呢？操作系统为进程提供服务，其中有些服务牵涉到安全。显然，在这种情况下，操作系统需要小心行事，在安全上做对的事。但之所以允许操作系统提供服务，是因为这些事有时确实需要做。所以，操作系统能够提供的任何服务，大概都应该提供，只是要在合适的情况下。

操作系统决定是执行某项服务，还是因为它会破坏安全目标而拒绝执行，一切都取决于上下文。在上下文中，最重要的因素恐怕是谁在提出请求。在现实世界里，如果你的另一半让你下班路上顺便去商店买一加仑牛奶，你多半会照办；要是街上的陌生人提同样的要求，你多半不会。在操作系统中，如果系统管理员要求操作系统安装一个新程序，操作系统大概应该照办；而如果是从某个随便什么网页上下载的脚本要求安装新程序，操作系统在安装之前就应该更加小心。在计算机安全的讨论中，我们常把提出请求的一方称为**主体**（principal）。主体是在安全上有意义、能够请求访问资源的实体，比如人类用户、用户组，或者复杂的软件系统。

所以，知道是谁在请求操作系统服务，对实现安全目标至关重要。操作系统怎么知道这一点呢？我们倒过来推一推，把它弄清楚。

请求操作系统服务，最常见的方式是由某个进程发起系统调用，从用户代码陷入操作系统。随后操作系统接管控制，为响应这次系统调用执行某项服务。发起调用的进程关联着一个由操作系统控制、描述该进程的数据结构，所以操作系统可以检查这个数据结构，确定进程的身份。根据这个身份，操作系统就有机会依据策略决定是否执行所请求的操作。在计算机安全的讨论中，代表主体发出请求的进程或其他活动的计算实体，常被称为该主体的**代理**（agent）。

请求是要访问某个特定的资源，我们通常把这个资源称为访问请求的**客体**（object）[1]。操作系统要么已经确定这个代理进程可以访问该客体，要么还没有。如果已经确定该进程获准访问，操作系统可以记住这个决定，剩下的只是记录这一事实，大概记在 PCB 这类每个进程一份的数据结构里。例如，我们在研究内存虚拟化时发现，页表这类每进程一份的数据

[1]这又是计算机科学对 object 一词的重载。这里它与“面向对象”无关，指的是更一般的概念：一个具有边界和行为的具体资源，比如一个文件或一个 IPC 通道。

结构，记录了进程在任一时刻可以访问哪些页和页帧。操作系统为记录这类访问决定、以备日后查阅而创建和管理的数据，无论是什么形式，常被称为**凭证**（credential）。

然而，如果操作系统还没有生成表明某个代理进程可以访问某个客体的凭证，它就需要知道该进程所属主体的身份，才能决定是否批准这个请求。不同的操作系统给主体用过不同类型的身份。例如，大多数操作系统都有用户身份的概念，这里的用户通常是某个人。（多年来，用户的概念不断扩展，变得更加强大，后面我们会看到。）这样，某个人运行的所有进程，也许都关联着同一个身份。另一种常见的身份类型是用户组。在一家制造企业里，你可能希望让所有销售人员都能访问库存信息，好让他们知道仓库里有多少零件和小玩意儿，而人力资源部门的人就没必要访问这些信息[2]。

还有一种身份形式是进程正在运行的程序。回想一下，进程就是运行中的程序。在有些系统中（比如 Android 操作系统），你可以把某些特权授予特定的程序。这些程序每次运行时都能使用这些特权，而其他程序不能。

不管我们根据哪种身份做安全决定，都必须有某种办法把这个身份附加到特定的进程上。显然，这种附加是一个至关重要的安全问题。如果你把一个程序员的进程误认成会计部门员工的进程，你的银行账户可能就被掏空了。（更别提还得再招一个程序员。）又或者，你的公司总裁要向投资人做一次重要的演示，而你没能正确识别他或她的身份，系统因此不让总裁访问那份本可以打动几位潜在投资人的演示文稿；等公司查明是你搞砸了下一轮创业融资，你可能就要丢掉饭碗了。

另一方面，除了操作系统自身的活动，一切都是由某个进程执行的。所以，只要我们能把进程的身份弄对，就可以相当有把握地在每个重要操作上检查策略。不过，我们需要记住操作系统通常的身份认证方式的另一个重要特点：主体一旦通过身份认证，系统几乎总是会在至少整个进程生命周期内都依赖这个认证结论。这一特点让“做对”变得格外重要，出了错不容易纠正。由此引出关键问题：

关键问题：如何安全地识别进程
对于支持属于多个主体的进程的系统，我们如何确保每个进程都附加了正确的身份？创建新进程时，如何确保新进程的身份是正确的？如何确保恶意实体无法不当地更改进程的身份？

54.2 给进程附加身份

进程是从哪里来的？通常是由其他进程创建的。所以，给新进程附加身份的一个简单办法，就是复制创建它的那个进程的身份：子进程继承父进程的身份。具体来说，当操作系统处理旧进程 A 创建新进程 B 的调用（例如 `fork`）时，它查看 A 的进程控制块，确定 A 的身份，为 B 创建一个新的进程控制块，然后把 A 的身份复制进去。很简单，对吧？

[2]还记得上一章讲的最小特权原则吗？这里就是它的一个应用实例。如果你没有给人力资源部门的员工下令清空仓库里所有 pop-doodle 的权限，那么即使有员工心怀不轨，也做不到这一点。阅读本书安全部分的各章时，留意一下我们前面讨论过的那些安全原则还有哪些其他应用。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

如果所有进程的身份始终都相同，这样做当然没问题。我们可以在操作系统启动时创建一个原始进程，也许给它分配某个不属于任何人类用户的特殊系统身份。其他所有进程都是它的后代，都继承这唯一的身份。可是，如果真的只有一个身份，我们就没法实施任何区分不同进程特权的策略。

我们必须让某些进程具有不同的身份，并利用这些差异来管理安全策略。考虑一个多用户系统。我们可以根据进程属于哪个人类用户来给它分配身份。如果我们的安全策略主要是规定某些人可以做某些事、另一些人不可以，那么我们现在就知道该怎么做决定了。

如果进程有一个与安全相关的身份，比如用户 ID，我们就得给新进程设置正确的用户 ID。在大多数系统中，用户平时都通过某个进程工作：在命令行系统中是 shell 进程，在窗口系统中是窗口管理器进程。你已经想到这两者本身也都得是进程了吧？所以，当你在 shell 里输入一条命令，或者在窗口系统里双击图标启动一个进程时，你就是在要求操作系统以你的身份启动一个新进程。

很好！不过我们还有一个问题要处理：那个 shell 或窗口管理器是怎么把你的身份附加到自己身上的？这时，操作系统的一点特权就派上用场了。用户刚开始与系统交互时，操作系统可以为这个用户启动一个进程。由于操作系统可以随意摆弄自己的数据结构，比如进程控制块，它就可以把这个新进程的所有者设为刚加入系统的那个用户。

这也没问题，但操作系统是怎么确定用户的身份，从而正确设置进程所有者的呢？你大概能猜到答案：用户登录了，也就是说，用户向操作系统提供了身份信息，证明了自己是谁。这样，我们就为操作系统找到了一个新要求：它必须能够向人类用户询问身份，并验证他们确实是自己所声称的那个人，这样才能给进程附加可靠的身份，进而用这些身份实施安全策略。在操作系统里，一件事往往会牵出另一件事。

那么，操作系统怎么做到这一点呢？显然，我们正在把一座后果难以预料的高耸安全大厦，建在操作系统能否在这里做出正确决定的基础上，所以这件事很重要。我们有哪些选择呢？

54.3 如何认证用户？

话说这个人走到一台计算机前……

先不管能编出什么笑话，我们能做些什么，让系统以合理的准确度确定这个人是谁？首先，如果这个人根本不是系统的授权用户，我们就应当彻底拒绝这次企图混进来的尝试。其次，如果他或她是授权用户，我们就需要确定：是哪一个？

传统上，认证人的身份有以下三种方式：

- 基于你知道什么来认证
- 基于你拥有什么来认证
- 基于你是什么来认证

这里说的“传统上”，指的是，嗯，真正古典意义上的传统，可以一直追溯到古希腊人和古罗马人。例如，公元前 2 世纪的波利比乌斯（Polybius）描述过罗马军队怎样用“口令”来区分敌我 [P-46]，这是基于你知道什么来认证的例子。公元 2 世纪的某个时候，一位名叫 Celer 的罗马建筑师为他的一个奴隶写了一封推荐信（这封信至今还在），让奴隶交给帝国

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

的一位代理官（procurator）[C100]，这是基于奴隶拥有什么来认证。再往前，在（字面意义上的）《圣经》时代，一场战斗之后，基列人要求逃难者说出“示播列”（shibboleth）这个词，因为他们要找的敌人（以法莲人）发不准这个词的音 [JB-500]。这是一种基于你是什么来认证的方式：你是以基列方言为母语的人，还是以以法莲方言为母语的人。

既然已经说明这些认证方法由来已久，我们就跳过几个世纪的历史，直接来到计算机时代，讨论如何在计算机认证中使用它们。

54.4 基于你知道什么来认证

基于你知道什么来认证，最常见的做法是使用密码。在计算机安全领域，密码有着漫长（而且基本上不怎么光彩）的历史，至少可以追溯到 20 世纪 60 年代初 MIT 的 CTSS 系统 [MT79]。密码是只有被认证的一方才知道的秘密。登录时，这一方向计算机的操作系统透露这个秘密，以此证明自己的身份。（你应该会想：这是否意味着系统也必须知道密码？这又会带来什么后果？我们后面会讲到。）显然，这种认证方式的效果取决于几个因素。我们假定其他人不知道这一方的密码，如果知道，系统就会被骗。我们还假定其他人也猜不到这个密码。当然，还要假定这一方自己必须知道（并且记得）密码。

我们先来解决其他人知道密码的问题。先不说猜测，他们还能怎么知道密码？已经知道密码的人可能会说漏嘴，所以必须知道密码的人越少，我们要担心的人就越少。我们要认证的那个人当然得知道密码，因为我们正是根据他知道密码来认证他的。我们确实不希望任何其他人能以这个人的身份通过系统认证，所以最好没有任何第三方知道密码。从广义上考虑“第三方”的含义，这也意味着用户不应该把密码写在纸条上，因为任何偷到纸条的人都会知道密码。但似乎还有一方需要知道密码：我们的系统本身。这又暗示了另一个可能的漏洞，因为系统保存的密码副本可能会泄露出去[3]。

不过，有意思的是，我们的系统其实并不需要知道密码。仔细想想系统在检查用户提供的密码时都在做什么。它是在检查用户是否知道密码，而不是密码究竟是什么。那么，如果用户把密码提供给我们，而我们并不知道密码，系统到底怎么才能做到这一点呢？

提示：避免存储秘密

存储明文密码或密钥这样的秘密是一件危险的事，因为秘密往往会泄露出去。如果不需要，就不要存储它们，以此保护你的系统。如果确实需要，就用强大的密码学散列函数，以散列后的形式存储它们。如果做不到，就用安全的密码算法加密它们。（也许你正在心里抱怨我们还没讲过这些。耐心点。）存储的地方要尽可能少，副本也要尽可能少。别忘了编辑器的临时文件、备份、日志之类的地方，秘密可能也藏在那里。记住，你嵌入到交给别人的可执行文件中的任何东西都不会是秘密，所以把秘密存在可执行文件里尤其危险。在某些情况下，甚至只保存在运行中程序堆里的秘密也被泄露过，所以即使在运行中的程序里，也要避免存储和保留秘密。

[3] “可能”这个词太轻了。已知最早的存储密码泄露事件发生在 1962 年 [MT79]；直到今天，这类泄露仍然以令人沮丧的频率发生，而且规模大得多。[KA16] 讨论了一次超过 1 亿个以可用形式存储的密码被泄露的事件。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

你其实已经知道答案了，至少一听到答案就会一拍脑门说："我早该想到的。"存储密码的**散列值**（hash），而不是密码本身。当用户提供他或她声称的密码时，对这个声称的密码做散列，然后与存储的散列值比较。如果相符，你就相信他或她知道密码；如果不符，就不相信。很简单，对吧？这样一来，你的系统就不需要存储真正的密码了。这意味着，即使你在存储认证信息时不够小心，你丢失的其实也不是密码，而只是它们的散列值。散列算法本质上是不可逆的，所以对手无法利用偷到的散列值得到密码。如果攻击者提供的是散列值而不是密码，系统会对这个散列值再做一次散列，而散列值的散列值是不会与散列值相符的。

不过事情还不止这么简单。存储密码的散列值，好处在于即使存储的副本泄露给了攻击者，攻击者也不知道密码本身。但仅仅存储一个与密码不同的东西还不够。我们还要确保，不管存储的是什么，都不会给攻击者猜测密码提供任何帮助。如果攻击者偷到了散列后的密码，他或她应当无法通过分析散列值得到任何关于密码本身的线索。有一类特殊的散列算法，称为**密码学散列函数**（cryptographic hash），它让人无法利用散列值推算出密码是什么，除非真的把猜测的密码送进散列算法去试。密码学散列函数有一个不幸的特点：它们很难设计，所以即使聪明人也不应该自己去设计，而是使用专家设计好的。这正是现代系统对密码做散列时应当做的：使用经过深入研究、没有已知缺陷的密码学散列函数。在不同的时期，满足这些要求的密码学散列算法可能各不相同。在撰写本文时，SHA-3 [B+09] 是美国的密码学散列算法标准，是一个不错的选择。

我们来看另一个问题：猜测。想冒充某个用户的攻击者，能不能直接猜出密码？考虑最简单的密码：只有一位，值为 0 或 1。如果你的密码只有一位长，攻击者猜"0"就有一半的机会猜对。即使猜错了，只要允许猜第二次，攻击者也就知道密码是"1"，从而猜对。

显然，一位的密码太容易猜了。8 位的密码怎么样？这样你可以选择的密码就有 256 种。如果攻击者猜 256 次，迟早会猜对，平均要猜 128 次。这比只需要猜两次要好，但还是不够好。到这里你应该清楚了，密码的长度对抵御猜测至关重要。密码越长，越难猜。

但还有另一个重要因素，因为我们通常希望人类用键盘之类的东西输入密码。既然我们已经认定把密码写下来不安全，那这个人就只能记住它。早期的密码为了解决这个问题，只允许使用字母。这样虽然更容易输入和记忆，却也大大减少了攻击者为找出某人密码而需要猜测的位模式数量，因为所有不代表字母的位模式都不会出现在密码里。随着时间推移，密码系统倾向于扩大密码中可用的字符范围，包括大小写字母、数字和特殊字符。可能性越多，越难猜。

所以，我们需要由多种不同类型的字符组成的长密码。但攻击者知道，人们不会用这些字符的随机组合做密码。人们常常选择名字或熟悉的单词，因为它们容易记。因此，试图猜测密码的攻击者会先尝试名字和单词列表，然后才尝试随机字符串。这种猜测密码的方式称为**字典攻击**（dictionary attack），它可能非常有效。这里的字典不是韦氏词典（甚至也不是《牛津英语词典》），而是一份专门的列表，收录了人们常用作密码的单词、名字、有意义的数字串（比如"123456"）以及其他字符模式，并按被选作密码的可能性排序。一次好的字典攻击可以破解一个典型网站 90% 的密码 [G13]。

如果你在设置系统时足够聪明，攻击者其实不应该能对登录进程发起远程字典攻击。只要稍加小心，攻击者在头五六次猜测中就猜不出用户的密码（唉，有时就是一点都不小心，攻击者也就猜中了）。而且没有任何正当理由让你的系统允许远程用户对一个账户的密码猜

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：密码库

避免选择密码这一难题的一种办法，是使用所谓的密码库（password vault）或钥匙串（key chain）。这是保存在你计算机上的一个加密文件，里面存着各种密码。它本身用一个密码加密。要从密码库里取出密码，你必须提供密码库本身的密码。这样，为每个网站记住一个不同密码的问题，就变成了只需记住一个密码。此外，它还能确保攻击者只有在既拥有打开密码库的那个特殊密码，又能访问密码库本身时，才能使用你的密码。当然，以这种方式安全地存储密码，好处受限于密码库中所存密码的强度，因为猜测和字典攻击仍然奏效。有些密码库会为你生成强密码，这些密码不太好记，但这无关紧要，因为需要记住它们的是密码库，而不是你。你也能找到把密码存到云端的密码库。然而，如果你把密码的明文版本提供给它们存储，你就是在和另一个其实并不需要知道这个密码的实体共享密码，从而承担了一个你或许不该承担的风险。如果云端只存储你加密后的密码，风险就要低得多。

上 15,000 次都猜不对。所以，你可以在某个账户的密码被猜错太多次时关闭对它的访问，或者（更好的办法）在猜错几次之后大幅放慢密码检查的速度（这会让一次漫长的字典攻击耗费不可行的时间），从而保护账户免受这类攻击。

但如果攻击者偷走了你的密码文件呢?既然我们假定你一直在认真听讲，文件里存的是密码的散列值，而不是密码本身。但我们同样假定，你也认真听了我们让你使用广为人知的密码学散列函数的建议，而你知道用的是哪个，偷走你密码文件的人也知道。如果攻击者拿到了你散列后的密码、散列算法、一部字典和一些算力，就可以慢慢地、一个一个地猜你的密码。更糟的是，如果每个人都使用同一个密码学散列算法（实际上他们很可能就是这样），攻击者只需把每个可能的密码送进散列函数算一次，把结果存起来（本质上就是把字典转换成了散列形式）。这样，当攻击者偷到你的密码文件时，他或她只需把你散列后的密码与这部新生成的散列字典做字符串比较，速度要快得多。

有一个简单的解决办法：在对新密码做散列并存入密码文件之前，先生成一个很大的随机数（比如 32 位或 64 位），把它拼接到密码上，对拼接结果做散列，再把散列值存起来。你还需要存储这个随机数，因为当用户尝试登录并提供正确的密码时，你需要把用户提供的内容与存储的随机数拼接起来，再送进散列算法。否则，单独对密码做散列，结果不会与你存储的值相符。通常，这个随机数（称为**盐**，salt）就存储在密码文件中，紧挨着散列后的密码。这个概念最早由 Robert Morris 和 Ken Thompson 在一篇关于密码安全的早期论文中提出 [MT79]。

这为什么有帮助?攻击者再也不能只为字典中的密码生成一份到散列值的转换表了。他需要为每一个可能的盐各生成一份转换表，因为被偷走的密码文件里，每个密码的盐很可能都不同。如果盐有 32 位，字典里的每个单词就要有 2^{32} 种不同的转换，这使得预先计算转换的办法不可行。字典攻击只能针对被偷的密码文件中的每一项，用该密码的盐对每个猜测重新做散列。如果你的密码选得不好，攻击仍然可行，但代价要高得多。

任何使用密码并重视安全的好系统，都会存储经过密码学散列并加了盐的密码。如果你的系统没有这样做，你就是在让用户冒险。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

使用密码还有其他一些令人担忧的问题，但其中很多并不是操作系统特有的，所以这里就不再往它们身上泼脏水了。只需说一句：计算机安全界普遍认为，密码是一种过时的技术，对今天的环境来说已经不够安全了。它们充其量只能作为协同使用的几种认证机制之一。这种思路称为**多因素认证**（multi-factor authentication），其中最广为人知的是**双因素认证**（two-factor authentication）。你可能已经熟悉这个概念：要从 ATM 取钱，你需要知道你的个人识别码（PIN），这本质上就是一个密码。但你还需要提供更多证明你身份的证据……

54.5 基于你拥有什么来认证

我们大多数人可能都遇到过需要出示身份证件才能进入某个地方的情况。至少，我们可能都参加过凭票入场的活动。这两种都是基于你拥有什么来认证的例子，在这里，你拥有的就是身份证件或门票。

向操作系统证明自己的身份时，情况就有些不同了。在一些特殊情况下，比如上面提到的 ATM，设备（毕竟它里面也有一台计算机，你知道的吧?）有专门的硬件来读取我们的 ATM 卡。这个硬件让它能确定：没错，我们确实有那张卡，从而在 PIN 之外提供了进一步的证明。大多数台式机、笔记本电脑、平板电脑、智能手机之类的设备都没有这种专门的硬件。那它们怎么知道我们拥有什么呢?

如果我们有某种能插到计算机某个端口上的东西，比如使用 USB 的硬件令牌，那么在合适的软件支持下，操作系统就能判断试图登录的用户有没有这个设备。有些安全令牌（有时被称为**加密狗**，dongle，这名字起得实在不怎么样）就是这样工作的。

在另一些情况下，既然我们反正是要认证一个人类用户，就可以借助这个人的能力，把信息从他或她拥有的东西传给需要认证的系统。例如，有些智能令牌会在一块很小的内置屏幕上显示一个数字或字符串。用户读出屏幕上的信息，用键盘输入计算机。操作系统并没有直接得到用户拥有这个设备的证据，但如果只有能拿到这个设备的人才可能知道该输入什么信息，这个证据也几乎一样可靠。

这类设备依赖于频繁更换它（直接或间接）传给操作系统的信息，也许每隔几秒换一次，也许用户每次尝试认证时都换。为什么? 因为如果不换，任何能从设备上获知这一静态信息的人，就不再需要这个设备也能冒充用户了。认证机制就从“你拥有的东西”变成了“你知道的东西”，其安全性也就取决于攻击者获知这个秘密的难度。

所有基于你拥有什么的认证方式都有一个弱点：如果你没带着它怎么办?如果你今天早上把智能手机落在梳妆台上了呢?如果你的加密狗在上班路上从口袋里滑出去了呢?如果一个手法高明的扒手在咖啡店里蹭了你一下，顺走了你的秘密认证设备呢?这下你有了双重麻烦。第一，你没有向操作系统证明身份所需的那件法宝。你可以对着计算机随便抱怨，它才不在乎。它会坚持要你拿出你弄丢的那件法宝。第二，别人拿到了你的法宝，可能就能冒充你，骗过依赖于基于你拥有什么来认证的操作系统。注意，我们前面提到的多因素认证在这里也能救你一命。如果小偷偷了你的安全令牌，却不知道你的密码，那么小偷还得先猜出密码，才能冒充你[4]。

[4]当然，前提是你没有用记号笔把密码写在被偷的那张智能卡背面。嗯，当时这看起来是个好主意……

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：Linux 的登录过程

Linux 沿袭早期 UNIX 系统的传统，基于密码认证用户，然后把这个身份绑定到与新登录用户关联的一个初始进程上，大致如上文所述。这里我们更详细地一步步描述，当用户走到键盘前尝试登录 UNIX 系统时，实际发生了什么，以此作为一个实际操作系统如何处理这一重要安全问题的具体例子。

1. 一个以特权系统身份运行的特殊 login 进程显示一个提示，要求用户输入身份，通常是一个简短的用户名。用户输入用户名并按回车。用户名会回显在终端上。
2. login 进程提示用户输入密码。用户输入密码，密码不回显。
3. login 进程在密码文件中查找用户提供的名字。如果找不到，login 进程就拒绝这次登录尝试。如果找到了，login 进程就确定内部用户标识符（一个唯一的用户 ID 号）、用户所属的组（另一个唯一的 ID 号）、登录完成后应提供给该用户的初始命令 shell，以及该 shell 应在哪个主目录中启动。此外，login 进程还会找到该用户密码的盐，以及正确密码加盐并散列后的版本，它们永久存储在系统中一个安全的地方。
4. login 进程把用户密码的盐与用户提供的密码组合起来，对组合结果做散列，再与上一步得到的存储版本比较。如果不相符，login 进程就拒绝这次登录尝试。
5. 如果相符，就 fork 出一个进程，把它的用户和组设为前面确定的值。login 进程的特权身份允许它这样做。然后切换到用户的主目录，并 exec 与该用户关联的 shell 进程（目录名和 shell 类型都是在第 3 步确定的）。

还有一些其他细节，涉及在这个用户退出后，确保同一个终端上还能让另一个用户登录，这里就不展开了。

注意，在第 3 步和第 4 步中，登录失败可能是因为系统里没有这个用户名，也可能是因为密码与用户名不符。如果登录失败，Linux 和大多数其他系统都不会指明是哪种情况。这样，攻击者就无法仅靠输入猜测来得知系统合法用户的名字，因为他们不知道自己猜的是一个不存在的名字，还是为一个合法用户名猜错了密码。不向未经认证的用户提供有用信息，一般来说是一种很好的安全思路，在其他类型的系统中也适用。

想一想，为什么 Linux 的登录过程回显用户输入的用户名，却不回显密码？回显用户名在安全上真的毫无坏处吗？回显用户名是绝对必要的吗？还是说这是用安全换取便利的一种权衡？为什么不回显密码？

如果你在实践中研究系统安全足够久，就会发现，学者（比如我）告诉你什么是安全的，和现实世界中实际发生的事情之间，存在着相当大的差距。差距的部分原因在于，现实世界需要处理现实的问题，比如用户的便利。另一部分原因在于，安全学者往往倾向于贬低任何他们能想出办法破坏的东西，即使那个办法本身并不怎么实际。在基于你拥有什么的认证机制方面，有一个例子是通过向用户手机发送短信来认证用户，用户再把短信内容输入计算机。从理论上讲，这听起来非常薄弱。除了手机丢失的危险，安全专家还喜欢设想一些奇特的攻击，比如短信被错发到攻击者的手机上，让攻击者能把短信里的秘密信息提供给计算机。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

在实践中，人们通常随身带着手机，也会合理地小心不弄丢它。即使弄丢了，他们也会很快发现，并同样迅速地采取措施解决问题。所以，从手机丢失到系统得知不能再用这部手机认证你，中间的时间窗口很可能相对较短。同样在实践中，重定向发往手机的短信是可能的，但远非轻而易举。至少在绝大多数情况下，攻击者为此付出的努力很可能超过他骗过认证系统所能得到的好处。所以，一种让安全纯粹主义者吓得不敢直视的机制，在实际使用中却能提供相当合理的安全性[5]。记住这个教训。即使考试不考[6]，它在你以后的职业生涯中某个时候也可能派上用场。

54.6 基于你是什么来认证

如果你不喜欢密码这类方法，也不喜欢给用户发智能卡或安全令牌，还有另一个选择。人类（这里我们要认证的正是人）是独一无二的生物，每个人都有与众不同的身体特征，有时差别细微，有时差别明显。除了人体的属性（从最底层的 DNA 到最表层的面容），人的行为也有一些特征是独一无二的，或者至少不会有很多人共有。这一观察表明，只要操作系统能准确地测量这些属性或特征，就能把一个人和另一个人区分开，从而解决认证问题。

这种方法对很多人都很有吸引力，尤其是对那些从没试过把它做出来的人。从这个基本观察出发，到做出一个能用、可靠的认证系统，远非易事。但它是可以做出来的，可靠程度和其他认证机制差不多。我们可以用它，但它不会完美，也有它自己的一系列问题和挑战。

记住，我们说的是由一个计算机程序（要么是操作系统本身，要么是操作系统为此调用的某个独立程序）来测量人的某个特征，并判断它是否属于某个特定的人。想想这意味着什么。如果我们打算用智能手机上的摄像头做人脸识别，来认证手机的主人呢？如果我们判定是对的人，就允许我们拍下的那个人使用这部手机。如果不是，就给他们（在网络意义上）喝个倒彩，把他们挡在外面。

你应该已经发现这里的几个难题了。首先，摄像头拍到的应该是正拿着手机的那个人。也许是机主，也许不是，拍照正是为了弄清这一点。如果不是，我们就应当假定，不管这人是谁，都想骗我们相信他就是真正的机主。如果这人长得很像正确的用户，但并不是呢？如果这人戴着面具呢？如果这人举着正确用户的照片，而不是露出自己的脸呢？如果光线很暗，或者这人没有正对着摄像头呢？反过来，如果确实是正确的用户，但他没有正对着摄像头，或者光线很暗，或者他的样子有了别的变化（比如发型）呢？

计算机程序识别人脸的方式和人不一样。它们处理数据的方式和程序一贯的做法相同：把数据转换成 0 和 1，再用某种算法处理。所以，你拍的那张“照片”实际上是一组数字，表示明暗、色调、对比度之类的信息。好，然后呢？该判断这是不是正确那个人的照片了！怎么判断？

如果是密码，我们可以存储正确的密码（或者更好的做法是存储正确密码的散列值），然后把输入的内容（或其散列值）与存储的值比较。完全相符就认证通过，否则就不通过。对于

[5]不过，2016 年，美国国家标准与技术研究院（NIST）发布了指南草案，至少在某些情况下，不再建议把这种技术用于双因素认证。这里又有一个安全上的教训：今天管用的东西，明天未必管用。

[6]我们不知道你怎么样，反正每次一提到“测验”“小测”或者“考试”，我们的心都会漏跳一两拍。当了太多年学生，人就会变成这样。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

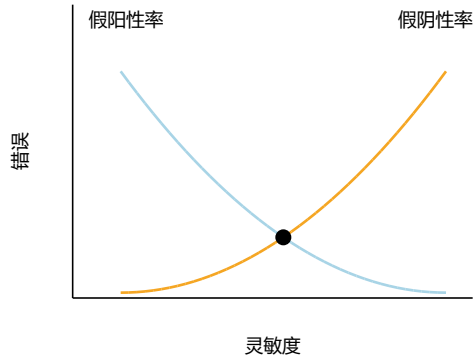


图 54.1：**交叉错误率**

代表我们刚拍下的照片的这一组 0 和 1，我们能不能也这样做？能不能把正确用户的一张照片永久存在某个文件里（同样以 0 和 1 的形式），然后把摄像头得到的数据与这个文件比较？

大概不能像比较密码那样比较。考虑我们刚才提到的一个因素：光线。如果文件中存储的照片是在明亮的光线下拍的，而摄像头拍出来的照片是在昏暗的光线下拍的，这两组 0 和 1 肯定不会相符。事实上，即使是在完全相同的条件下、间隔一秒拍的同一个人的两张照片，也几乎不可能由完全相同的一组位来表示。所以，显然我们不能按逐位相等来比较。

我们需要在更高的层次上分析两张照片，一张是存储的正确用户的照片，一张是刚拍下的、声称是该用户的那个人的照片，再基于分析结果来比较。一般来说，这需要从照片中提取更高层的特征，然后比较这些特征。例如，我们可能会尝试计算鼻子的长度，或者确定眼睛的颜色，或者为嘴的形状建立某种模型。然后，我们比较两张照片的同一组特征。

然而，即使这样，也不太可能完全相符。例如，光线可能会让人感知到的眼睛颜色略有变化。所以，我们在比较时需要允许一些马虎。如果特征“足够接近”，就认证通过；否则就不通过。我们要找的是相近的匹配，而不是完全的匹配，这就像让骆驼把鼻子伸进了帐篷：容差从此进了我们的认证机制。如果除了最接近的匹配之外一概不容，那么总有些日子，我们无法把真正用户的照片与存储的版本匹配起来。这称为**假阴性**（false negative），因为我们错误地决定不予认证。如果我们对测量数据与存储数据之间的差异过于宽容，就会认证一个并不是他所声称的那个人的用户。这称为**假阳性**（false positive），因为我们错误地决定予以认证。

生物识别的本性决定了，任何实现都会有其特有的假阳性率和假阴性率。两者都不好，所以你希望两者都低。对于某种生物识别认证技术的任一具体实现，你通常可以把它调到某个假阳性率，或者调到某个假阴性率，但通常无法让两者同时降到最低。假阳性率下降，假阴性率就会上升，反之亦然。**灵敏度**（sensitivity）描述的是匹配必须有多接近。

图 54.1 展示了这两种错误率之间的典型关系。注意两条曲线交点处的那个圆点。这个点代表**交叉错误率**（crossover error rate），是描述生物识别准确度的一个常用指标。它代表两种错误之间的对等权衡。人们并不总是把生物识别系统调到交叉错误率上，因为你可能

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

更在意其中一种错误。例如，一部智能手机如果因为不喜欢今天读到的指纹而经常把合法用户锁在外面，是不会受欢迎的，而偷了这部手机的小偷恰好有相似指纹的可能性很低。在这种情况下，也许低假阴性更重要。另一方面，如果你用视网膜扫描来打开银行金库，偶尔要求银行经理再扫描一次并不算太糟，而让劫匪用一只假眼打开金库就是一场灾难了。在这种情况下，低假阳性可能更好。

先不说用生物识别认证的可靠性问题，用人的特征来认证还有一个大问题：很多测量这些特征的技术都需要专门的硬件，而大多数机器上不太可能有。很多计算机（包括智能手机、平板电脑和笔记本电脑）都可能有摄像头，但嵌入式设备和服务器多半没有。有指纹读取器的机器相对较少，能测量更奇特的生物特征的机器就更少了。虽然有少数几种生物识别技术（比如测量打字模式）需要的硬件相对常见，很多机器上本来就有，但这样的技术并不多。即使有专门的硬件设备，把它们用于这个目的的便利程度也可能有限。

考虑使用生物识别认证时，还有一个问题值得思考：测量生物特征量的地方和检查它的地方之间，是否存在物理上的间隔。特别是，检查不受信任的机器通过网络提供的生物识别读数是很危险的。通过网络传来的，只不过是分散在一条或多条消息中的一组位，不管它代表的是网页的一部分、VoIP 通话中的一个音素，还是扫描指纹的一部分。位就是位，任何人都能制造出任何他想要的位模式。如果远程的对手知道代表你指纹的位模式是什么样子，他们也许既不需要你的手指，也不需要指纹扫描仪，就能制造出这个位模式，并把它送进你的机器。当执行扫描的硬件物理连接在你的机器上时，混入一个并非来自该设备的伪造位模式的机会就少一些。当硬件在世界的另一端、在一台你无法控制的机器上时，机会就多得多了。这里的要点是：对远程提供给你的生物识别认证信息要小心。

总的来说，听起来生物识别好像非常不适合用于认证，但这个结论是错的。说到底，前面几节可能也让所有认证方法听起来都很糟糕。它们当然都不完美，但作为系统设计者，你的任务不是找到完美的认证机制，而是使用适合你的系统及其环境的机制。一部智能手机内置一个好的指纹读取器，可能就能很好地完成任务。一个又长又猜不到的密码，可以提供相当程度的安全性。设计良好的智能卡，可以让你在不握着它的情况下几乎不可能通过认证。而在每种机制失效的地方，你也许可以再用第二种或第三种在不同情况下才会失效的认证机制来弥补。

54.7 认证非人类主体

不，我们说的不是外星人或异次元生物，甚至也不是你的猫。如果你从广义上考虑今天计算机的使用方式，就会发现在很多情况下，正在运行的进程并没有关联任何人类用户。想想 Web 服务器。实际上并没有某个人类用户登录，需要把他的身份附加到 Web 服务器上。再想想嵌入式设备，比如智能灯泡。没人会登录到一个灯泡上，但灯泡里肯定运行着代码，而且很可能是面向进程的代码。

从机制上说，操作系统处理这类进程的身份并不困难。只要在相关系统上建立一个名为 `webserver` 或 `lightbulb` 的用户，再把这个“用户”的身份附加到负责运行 Web 服务器或开关灯泡的那些进程上就行了。但这确实会引出一个问题：怎样确保只有真正的 Web 服务器进程才带有这个身份？我们可不希望 Web 服务器机器上的某个随便什么用户创建出看起来属于服务器、而不属于该用户的进程。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

一种办法是也给这些非人类用户设置密码。只要给 Web 服务器用户分配一个密码就行了。什么时候用它？需要的时候用，也就是当你想创建一个属于 Web 服务器的进程，而当前还没有这样的进程时。系统管理员可以以 Web 服务器用户的身份登录，得到一个命令 shell，再用它生成服务器开展业务所需的那些进程。和往常一样，这个 shell 进程创建的进程会继承父进程的身份，在这里就是 `webserver`。更常见的做法是跳过中间环节（这里就是登录），提供某种机制，允许特权用户创建不属于自己、而属于 `webserver` 等其他用户的进程。或者，我们可以提供一种机制，允许进程更改自己的所有者，这样 Web 服务器进程可以先以其他某个用户（比如系统管理员）的身份启动，然后把所有者改成 `webserver`。还有一种办法是允许临时更改进程身份，同时记住原来的身份。（我们会在后面的章节中更多地讨论最后这种办法。）显然，这些办法中的任何一种都需要严格的控制，因为它们允许一个用户创建属于另一个用户的进程。

如上所述，确定一个进程能否分配给这类非人类用户时，最常用的认证方法是密码。不过，有时根本不需要认证非人类用户，而是赋予某些其他用户（比如受信任的系统管理员）一项权利：他们可以给自己创建的进程分配新的身份，而不需要提供除他们自己的认证信息之外的任何其他认证信息。在 Linux 和其他 UNIX 系统中，`sudo` 命令就提供了这种能力。例如，如果你输入以下命令：

```
sudo -u webserver apache2
```

这表示 `apache2` 程序应当以 `webserver` 的身份启动，而不是以运行 `sudo` 命令的那个人的身份启动。这条命令可能会要求运行它的用户提供自己的认证凭证（以便更加确定确实是特权用户在发出请求，而不是趁特权用户喝咖啡时动用计算机的什么路人），但不

补充：其他认证方式

通常，你知道什么、你拥有什么和你是什么，已经涵盖了有用的认证方式，但有时也有其他选择。想象你去车管所申请驾照。你大概会走到一个柜台前，和柜台后面的某个工作人员交谈，也许会给这个人一大堆个人信息，甚至付钱交驾照的费用。你到底凭什么相信这个人真的是车管所的员工，能给你办一张合法的驾照呢？你大概并不认识这个人；没人给你看官方证件；这个人也没有念出证明他或她是该机构内部人员的什么车管所秘密口诀。你相信这个人，是因为这个人站在某个特定的柜台后面，而那正是车管所员工所站的柜台。你是根据位置来认证这个人的。

偶尔，这种办法在计算机系统中也很有用，最常见于移动计算或普适计算。如果你想用它，就要仔细想想，你是怎样获得证据，证明对象真的在某个特定地点的。这其实相当棘手。

还有什么？也许有时你可以根据某人做了什么来认证。如果你关注的是个人特有的行为，比如打字模式或命令之间的延迟，那就是生物识别的一种。（例如，Google 就在它的 Android 手机中引入了这类多因素认证。）但你可能不那么在意认证他们究竟是谁，而更在意认证他们是否属于“行为良好的用户”这个集合。例如，很多网站不太关心访客是谁，而更关心他们是否正当地使用网站。在这种情况下，你可以根据他们与系统持续的交互，来认证他们属于这个集合。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

会要求提供与 webserver 关联的认证信息。当然，apache2 创建的任何子进程都会继承 webserver 的身份。我们会在访问控制一章中更多地讨论 sudo。

我们前面提到过的最后一个身份问题是：有时我们希望识别的不只是单个用户，而是具有共同特征（通常是与安全相关的特征）的用户组。例如，我们可能有四五个系统管理员，其中任何一个都可以启动 Web 服务器。与其把这项特权分别关联到每个人身上，不如创建一个在系统上有意义、具有这项特权的用户组，然后指明这四五个管理员是这个组的成员。这种组是与安全相关的主体的另一个例子，因为我们将根据组成员身份、而不是个人身份来做决定。当某个系统管理员想做需要组成员身份的事情时，我们就检查他或她是否是该组的成员。我们可以给每个进程关联一个组成员身份，也可以用进程的个人身份信息作为索引，去查一份记录人们所属组的列表。后一种方式更灵活，因为它允许我们把每个用户放进任意数量的组里。

大多数现代操作系统，包括 Linux 和 Windows，都支持这类组，因为它们让安全策略的应用变得简单灵活。它们处理组成员身份和组特权的方式，与处理个人身份和个人特权的方式大体类似。例如，子进程通常与父进程具有相同的组相关特权。使用这类系统时，重要的是要记住，组成员身份为用户提供了获取资源访问权的第二条途径，这既有好处，也有危险。

54.8 小结

如果我们想对系统中进程执行的操作应用安全策略，就需要知道进程的身份，才能做出正确的决定。整条进程链的起点，是在启动时创建一个属于某个系统用户的进程，它的用途就是认证用户。用户登录时，以一种或多种形式提供认证信息，证明自己的身份。系统用这些信息验证用户的身份，并把这个身份分配给一个新进程，让用户能够开展工作，这通常涉及运行其他进程。那些其他进程会从父进程那里继承用户的身份。特殊的安全机制可以允许更改进程的身份，或者把进程的身份设为与父进程不同的身份。这样，系统就能确信进程属于正确的用户，并据此做出安全决定。

无论从历史上还是从实践上看，提供给系统的认证信息，要么是认证用户知道的东西（比如密码或 PIN），要么是用户拥有的东西（比如智能卡或对一部智能手机的持有证明），要么是用户本身是什么（比如用户的指纹或声纹扫描）。每种方式各有优缺点。使用多因素认证可以获得更高程度的安全性，它要求用户提供不止一种形式的证据，比如既要提供密码，又要提供一个发送到用户智能手机上的一次性验证码。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

参考文献

[B+09] "The road from Panama to Keccak via RadioGatun" by Guido Bertoni, Joan Daemen, Michael Peeters, Gilles Van Assche. *The authors who developed SHA-3. For a more readable version, try the Wikipedia page first about SHA-3. There, you learn about the "sponge construction", which actually has something to do with cryptographic hashes, and not the cleaning of your kitchen.*

[C100] "Letter of recommendation to Tiberius Claudius Hermeros" by Celer the Architect. Circa 100 A.D.. *This letter introduced a slave to the imperial procurator, thus providing said procurator evidence that the slave was who he claimed to be. Read the translation at the following website* http://papyri.info/ddbdp/c.ep.lat;;81.

[G13] "Anatomy of a hack: even your 'complicated' password is easy to crack" by Dan Goodin. http://www.wired.co.uk/article/password-cracking, May 2013. *A description of how three experts used dictionary attacks to guess a large number of real passwords, with 90% success.*

[JB-500] "Judges 12, verses 5-6" The Bible, roughly 5th century BC. *An early example of the use of biometrics. Failing this authentication had severe consequences, as the Gileadites slew mispronouncers, some 42,000 of them according to the book of Judges.*

[KA16] *VK.com Hacked! 100 Million Clear Text Passwords Leaked Online* by Swati Khandelwal. http://thehackernews.com/2016/06/vk-com-data-breach.html. *One of many reports of stolen passwords stored in plain-text form.*

[MT79] "Password Security: A Case History" by Robert Morris and Ken Thompson. Communications of the ACM, Vol. 22, No. 11, 1979. *A description of the use of passwords in early Unix systems. It also talks about password shortcomings from more than a decade earlier, in the CTSS system. And it was the first paper to discuss the technique of password salting.*

[M+02] "Impact of Artificial "Gummy" Fingers on Fingerprint Systems" by Tsutomu Matsumoto, Hiroyuki Matsumoto, Koji Yamada, and Satoshi Hoshino. SPIE Vol. #4677, January 2002. *A neat example of how simple ingenuity can reveal the security weaknesses of systems. In this case, the researchers showed how easy it was to fool commercial fingerprint reading machines.*

[P-46] "The Histories" by Polybius. Circa 146 B.C.. *A history of the Roman Republic up to 146 B.C. Polybius provides a reasonable amount of detail not only about how the Roman Army used watchwords to authenticate themselves, but how they distributed them where they needed to be, which is still a critical element of using passwords.*

[TR78] "On the Extraordinary: An Attempt at Clarification" by Marcello Truzzi. Zetetic Scholar, Vol. 1, No. 1, p. 11, 1978. *Truzzi was a scholar who investigated various pseudoscience and paranormal claims. He is unusual in that he insisted that one must actually investigate such claims before dismissing them, not merely assume they are false because they conflict with scientific orthodoxy.*

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)